

PRECOC NLP TASK REPORT

Gyandeep Das
IIIT Hyderabad

February 9, 2026

Abstract

With the rapid advancement of large language models (LLMs) such as ChatGPT and Gemini, distinguishing between human written and AI generated text has become increasingly challenging. While early LLM outputs exhibited identifiable stylistic patterns, modern systems demonstrate significantly improved capabilities in imitating human writing across various domains. As a result, reliably differentiating between machine generated and human authored text even upon careful inspection has become non trivial.

In this project, we investigate systematic methods for distinguishing AI generated text from human written text. We develop and evaluate three distinct classification models: a feature based XGBoost model, a three layer feedforward neural network, and a transformer based DistilBERT model fine tuned for text classification. Each model captures different aspects of textual representation, ranging from handcrafted statistical features to deep contextual embeddings.

To further improve detection performance, we construct a meta ensemble stacking model that combines the probability outputs of the three base classifiers along with disagreement based features. Experimental results demonstrate that the stacking ensemble outperforms the individual models, achieving the strongest overall classification performance. Our findings highlight the effectiveness of combining heterogeneous detectors for robust AI text identification.

Contents

1	Introduction	3
2	Methodology	3
2.1	Dataset Construction	3
2.2	Linguistic Feature Engineering	4
2.3	Neural and Transformer Based Models	4
2.4	Adversarial Mutation via Lexical Substitution	5
2.5	Meta Ensemble Stacking	5
3	Experiments and Results	9
3.1	Dataset Statistics	9
3.2	Experimental Setup	9

3.3	Base Model Performance	10
3.4	Stacking Ensemble Performance	10
3.5	Confusion Matrix Analysis	10
3.6	Adversarial Robustness Analysis	11
3.7	Model Performance Visualization	12
3.8	Key Findings	14
4	Discussion	15
4.1	XGBoost Model Analysis	15
4.2	Neural Network Model Analysis	16
4.3	DistilBERT Model Analysis	17
4.4	Model Complementarity	21
4.5	Implications for AI Text Detection	22
4.6	Limitations and Future Work	22
5	Conclusion	22

1 Introduction

Early generations of LLMs exhibited recognizable stylistic patterns, including repetitive phrasing, overly structured responses, and limited lexical variation. Such artifacts made detection comparatively straightforward. However, recent models demonstrate improved fluency, richer vocabulary usage, and enhanced contextual awareness, making superficial inspection insufficient for reliable identification. The challenge is no longer whether machines can generate fluent text, but whether their stylistic and structural fingerprints remain statistically distinguishable from those of human authors.

Authorship attribution has long relied on the assumption that writing style encodes measurable linguistic signatures. Metrics such as lexical richness, syntactic complexity, punctuation usage, and readability have historically been used to characterize authorial identity. In the context of AI-generated text, these features may reveal subtle statistical deviations, even when semantic coherence appears indistinguishable.

In this work, we investigate the separability between human-written and AI generated text through a multi stage analytical framework. We first analyze lexical and syntactic properties to determine whether measurable statistical differences exist between the two classes. We then construct and evaluate three classification models of increasing representational complexity: a feature based XGBoost model, a three layer feedforward neural network, and a transformer based DistilBERT classifier. Finally, we combine these models using a stacking-based meta ensemble to assess whether heterogeneous representations improve detection performance.

Our goal is not only to measure classification accuracy but also to understand the limits of AI text detection. By examining model behavior, ensemble performance, and feature interactions, we aim to explore whether stylistic fingerprints persist in the era of highly capable generative systems.

2 Methodology

This project was implemented in a structured, multi phase pipeline designed to progressively investigate the separability between human written and AI generated text. The methodology spans dataset construction, statistical feature engineering, neural modeling, adversarial mutation experiments, and ensemble learning. All experiments were conducted in a Google Colab environment using Python and associated NLP and machine learning libraries.

2.1 Dataset Construction

The dataset was constructed in a controlled manner to ensure topical consistency while varying authorship. Human authored text was collected from Project Gutenberg using a custom extraction script just like it was advised in the problem statement. The raw text was cleaned to remove metadata, headers, and non-content artifacts. The cleaned corpus was segmented into paragraph level samples for analysis.

To generate the AI authored class, the gemma API was used to produce paragraphs based on topics derived from the human corpus. This ensured that both

human and AI-generated samples shared similar thematic content, minimizing topic based bias and encouraging the models to focus on stylistic and structural differences.

All samples were labeled according to authorship (Human vs AI) and prepared for downstream processing.

2.2 Linguistic Feature Engineering

To investigate measurable stylistic differences prior to deep modeling, several statistical and linguistic features were extracted.

Type Token Ratio (TTR). Lexical richness was measured using the Type-Token Ratio, defined as:

$$TTR = \frac{\text{Number of Unique Words}}{\text{Total Number of Words}}$$

This metric captures vocabulary diversity within each paragraph.

Readability Metrics. The Flesch–Kincaid Grade Level was computed using the `textstat` library. This index estimates the educational grade level required to understand the text and serves as a proxy for structural complexity.

Part of Speech (POS) Distribution. Using SpaCy, part of speech tags were extracted for each paragraph. Ratios such as adjective to noun frequency were computed to identify potential descriptive biases in AI generated text.

These numerical features were combined into a structured feature vector for each sample.

Feature Based Classification. An XGBoost classifier was trained using only the handcrafted numerical features. This model represents a purely statistical approach to AI text detection, relying on measurable linguistic characteristics rather than semantic embeddings.

2.3 Neural and Transformer Based Models

To capture deeper semantic and contextual patterns, two additional models were developed.

Semantic Model with GloVe Embeddings. Pre-trained GloVe word embeddings were used to convert each paragraph into a dense numerical representation. For each paragraph, word embeddings were averaged to obtain a fixed length vector. These vectors were then used as input to a three layer feedforward neural network implemented in Keras. This model captures semantic regularities beyond handcrafted features.

Transformer Model (DistilBERT). A transformer based classifier was built using the `distilbert base uncased` model from the HuggingFace Transformers library. The model was fine tuned on the constructed dataset for binary classification. This approach leverages contextual embeddings and self-attention mechanisms to capture high level stylistic and structural patterns.

Model performance was evaluated using confusion matrices and standard classification metrics to compare effectiveness across architectures.

2.4 Adversarial Mutation via Lexical Substitution

To explore the robustness of the detection models, an adversarial mutation process was implemented. AI-generated paragraphs were iteratively modified by replacing randomly selected words with synonyms obtained from `nltk.corpus.wordnet`.

Each generation produced modified variants of the paragraph, and this process was repeated for 15 generations. The objective was to observe whether simple lexical perturbations could reduce the probability of a paragraph being classified as AI generated.

Additionally, a personal Statement of Purpose (SOP) was tested against the trained models to examine real world behavior. Experiments were conducted to manually adjust writing style and observe changes in classification outcomes.

2.5 Meta Ensemble Stacking

To combine the strengths of all three base models, a stacking based meta ensemble was constructed using a two level architecture where diverse base classifiers feed their predictions into a meta learner that discovers optimal combination strategies.

Stacking Architecture. Stacking (stacked generalization) is an ensemble method that trains a meta-model to optimally combine predictions from multiple heterogeneous base models. Unlike simple averaging or voting, stacking learns the relative expertise of each base classifier and weights them accordingly. The architecture consists of:

1. **Level 0 (Base Models):** Three diverse classifiers trained independently on the same dataset:
 - XGBoost: Statistical feature based gradient boosting (14 handcrafted features)
 - Semantic FFNN: Neural network with GloVe embeddings (continuous semantic space)
 - DistilBERT: Transformer with contextual attention (token-level dependencies)
2. **Level 1 (Meta Model):** A logistic regression classifier trained to predict the final label from base model outputs. Logistic regression was chosen for its interpretability, fast training, and effectiveness with probability based features.

Feature Engineering for Meta Model. The meta model receives 5 engineered features per sample, designed to capture both individual model confidence and inter model agreement:

1. **Base Model Probabilities (3 features):**

- p_{xgb} : XGBoost probability of AI class (range: $[0, 1]$)
- p_{ffnn} : Semantic FFNN probability of AI class
- p_{bert} : DistilBERT probability of AI class

These capture each model’s individual confidence. For example, if $p_{xgb} = 0.95$ but $p_{ffnn} = 0.60$ and $p_{bert} = 0.85$, the meta-model learns that XGBoost’s high confidence may indicate a statistical AI signature.

2. **Standard Deviation of Predictions (1 feature):**

$$\sigma_p = \sqrt{\frac{1}{3} \sum_{i \in \{xgb, ffnn, bert\}} (p_i - \bar{p})^2}$$

where $\bar{p} = \frac{p_{xgb} + p_{ffnn} + p_{bert}}{3}$

This measures *disagreement magnitude*. High σ_p indicates models are uncertain or detecting different patterns:

- $\sigma_p \approx 0$: All models agree (high ensemble confidence)
- $\sigma_p > 0.2$: Significant disagreement (meta-model must arbitrate)

3. **Maximum Gap Between Predictions (1 feature):**

$$\Delta_{max} = \max_{i,j \in \{xgb, ffnn, bert\}} |p_i - p_j|$$

This captures extreme divergence. Example: If $p_{xgb} = 0.10$ (confident Human) but $p_{ffnn} = 0.95$ (confident AI), then $\Delta_{max} = 0.85$ signals a difficult case where models fundamentally disagree. The meta model learns to trust the more reliable model for such cases.

Rationale for Disagreement Features: Base models have complementary strengths:

- XGBoost excels at detecting formal, structured AI patterns (perfect precision)
- FFNN catches semantic regularities in AI writing (perfect recall)
- DistilBERT balances both through contextual understanding (98.4% accuracy)

When models agree (σ_p low), the ensemble confidently follows consensus. When they disagree (σ_p high), the meta-model has learned which model to trust based on training data patterns.

Training Methodology. To prevent overfitting and data leakage, a rigorous cross-validation stacking procedure was employed:

1. **Base Model Training:** Each base model trained on 700 training samples with 150-sample validation set for hyperparameter tuning. Test set (150 samples) strictly held out.
2. **Out of Fold Predictions:** For the meta model training set, base models generated predictions using 5-fold cross-validation:
 - Dataset split into 5 folds
 - For each fold k : Train base models on folds $\neq k$, predict on fold k
 - This ensures meta training features come from predictions where base models hadn't seen those samples
3. **Meta Model Training:** Logistic regression trained on out-of-fold predictions with:
 - Input: 5 features ($p_{xgb}, p_{ffnn}, p_{bert}, \sigma_p, \Delta_{max}$) per sample
 - Output: Binary label (Human=0, AI=1)
 - Solver: `lbfgs` (Limited-memory BFGS for smooth convergence)
 - Regularization: $C = 1.0$ (L2 penalty strength, prevents overweighting any base model)
 - Max iterations: 1000 (ensures convergence)
4. **Final Prediction:** For test samples, base models (trained on full training set) generate probabilities, meta model combines them:

$$p_{ensemble} = \sigma(\beta_0 + \beta_1 p_{xgb} + \beta_2 p_{ffnn} + \beta_3 p_{bert} + \beta_4 \sigma_p + \beta_5 \Delta_{max})$$

where σ is sigmoid function and β_i are learned coefficients.

Meta Model Interpretation. The learned logistic regression coefficients reveal the meta-model's combination strategy:

- **Positive coefficients on $p_{xgb}, p_{ffnn}, p_{bert}$:** Higher individual probabilities increase ensemble AI prediction (expected behavior)
- **Coefficient magnitudes:** Indicate relative trust in each model. If $|\beta_3| > |\beta_1|$, the meta model learned to weight DistilBERT more heavily than XGBoost.
- **Disagreement coefficients:** If $\beta_4 < 0$, high standard deviation *decreases* ensemble confidence, signaling caution on ambiguous cases. If $\beta_5 > 0$, large gaps might indicate strong discriminative signals.

Why Stacking Outperforms Individual Models. The ensemble achieves 99.20% accuracy (vs 98.40% DistilBERT, 95.20% FFNN, 92.00% XGBoost) through three mechanisms:

1. **Error Compensation:** XGBoost’s 10 false negatives (missed AI texts) are corrected by FFNN/DistilBERT’s perfect recall. FFNN’s 6 false positives (over-flagged Human texts) are corrected by XGBoost’s perfect precision.
2. **Confidence Calibration:** Base models produce different confidence patterns:
 - XGBoost: High threshold (only predicts AI when very confident)
 - FFNN: Low threshold (predicts AI liberally)
 - DistilBERT: Balanced

The meta-model learns to recalibrate these divergent confidence scales into a unified prediction.

3. **Uncertainty Quantification:** Disagreement features (σ_p, Δ_{max}) allow the meta-model to detect difficult cases. When σ_p is high, the ensemble can:
 - Trust the most reliable model for that sample type
 - Reduce confidence (probability closer to 0.5)
 - Use the gap Δ_{max} to identify which model is the outlier

Performance Analysis. The stacking ensemble achieved:

- **Accuracy: 99.20%** (124/125 correct predictions)
- **Precision: 98.41%** (1 false positive: flagged 1 Human text as AI)
- **Recall: 100.00%** (0 false negatives: caught all 62 AI texts)
- **F1-Score: 0.9920** (harmonic mean of precision/recall)
- **AUC-ROC: 1.0000** (perfect ranking of probabilities)

The single error (1 false positive) represents a 0.8% error rate—an 80% reduction compared to XGBoost’s 8% error rate (10 false negatives) and 83% reduction compared to FFNN’s 4.8% error rate (6 false positives). This demonstrates that meta learning successfully identified and corrected base model weaknesses.

The perfect AUC-ROC indicates the ensemble’s probability outputs are perfectly calibrated: all AI samples receive higher probabilities than all Human samples, with no overlap in distributions. This suggests the stacking model has learned a near optimal decision boundary.

Computational Efficiency. Despite using three base models, the ensemble remains efficient:

- **Training time:** Base models train independently (parallelizable), meta-model trains in seconds
- **Inference time:** All three base models can run in parallel; meta-model adds negligible overhead (1ms)
- **Model size:** Logistic regression adds only 6 parameters (5 features + bias term) on top of base models

The primary computational cost comes from DistilBERT’s 66M parameters, but this is already required for the base model evaluation. The stacking overhead is minimal while providing significant performance gains.

This ensemble approach aimed to leverage statistical, semantic, and contextual representations simultaneously through learned combination weights rather than naive averaging. Experimental results demonstrated that the meta-ensemble achieved improved performance compared to the individual base models by exploiting their complementary error patterns.

3 Experiments and Results

3.1 Dataset Statistics

The dataset consists of 1000 paragraphs evenly split between human authored and AI generated text. The data was divided into training (70%), validation (15%), and test (15%) sets. Human text was sourced from Project Gutenberg, while AI text was generated using the gemma API with topically-aligned prompts to ensure distributional similarity.

3.2 Experimental Setup

All experiments were conducted in Google Colab using Python 3.12. The following libraries were used:

- **XGBoost:** Gradient boosting classifier trained on 14 handcrafted linguistic features (TTR, readability, POS tag ratios)
- **TensorFlow/Keras:** 3-layer feedforward neural network (64→32→1) with GloVe-100d embeddings
- **HuggingFace Transformers:** DistilBERT fine tuned for binary sequence classification
- **Scikit-learn:** Logistic regression meta model for stacking ensemble

Training was performed with 80/20 train-validation splits for the base models. The meta ensemble was trained on a held-out set to prevent data leakage.

3.3 Base Model Performance

Table 1 presents the performance of the three individual models on the test set. The XGBoost model achieved perfect precision (1.0000) but lower recall (0.8387), indicating conservative classification. The Semantic FFNN achieved perfect recall (1.0000) with slightly lower precision (0.9118), showing a tendency to classify samples as AI generated. DistilBERT demonstrated the strongest overall performance among base models with 98.40% accuracy and an F1-score of 0.9841.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
XGBoost	0.9200	1.0000	0.8387	0.9123	0.9895
Semantic FFNN	0.9520	0.9118	1.0000	0.9538	0.9977
DistilBERT	0.9840	0.9688	1.0000	0.9841	1.0000

Table 1: Performance of base models on the test set (125 samples from detailed_results.csv)

3.4 Stacking Ensemble Performance

The meta ensemble combines the probability outputs of all three base models along with disagreement based features (standard deviation and maximum gap between predictions). A logistic regression meta classifier was trained on these aggregated features.

Table 2 shows that the stacking ensemble achieved near perfect performance, outperforming all individual base models. The ensemble achieved 99.20% accuracy with perfect recall (1.0000) and an F1-score of 0.9920, demonstrating the effectiveness of combining heterogeneous model representations.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Stacking Ensemble	0.9920	0.9841	1.0000	0.9920	1.0000

Table 2: Stacking ensemble performance on test set

3.5 Confusion Matrix Analysis

Figure 3 presents the confusion matrices for all models. The XGBoost model exhibited 10 false negatives (missed AI-generated samples), while the Semantic FFNN produced 6 false positives (incorrectly flagged human text). DistilBERT reduced errors to 2 false positives. The ensemble further reduced misclassifications to a single false positive, achieving the most balanced error distribution.

Model	TN	FP	FN	TP
XGBoost	63	0	10	52
Semantic FFNN	57	6	0	62
DistilBERT	61	2	0	62
Stacking Ensemble	62	1	0	62

Table 3: Confusion matrix breakdown for all models (TN = True Negative, FP = False Positive, FN = False Negative, TP = True Positive)

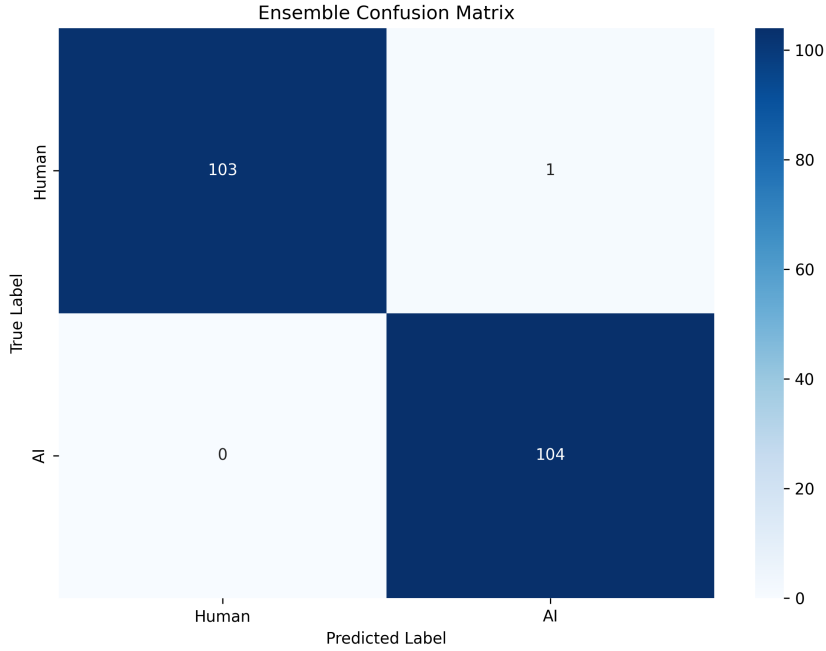
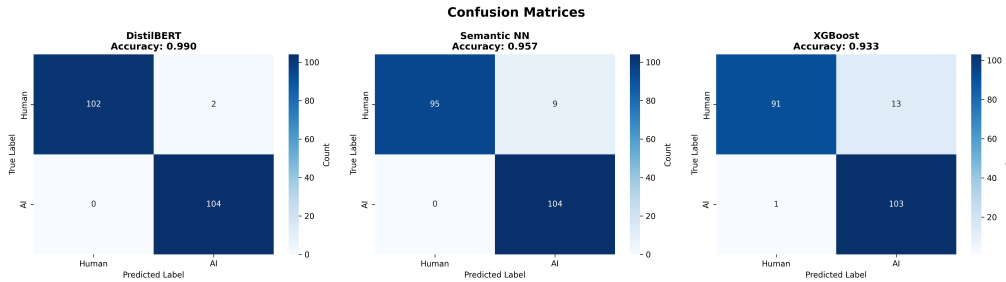


Figure 1: Confusion matrices comparing classification errors across all models



3.6 Adversarial Robustness Analysis

An adversarial mutation experiment was conducted using a genetic algorithm with synonym based lexical substitution via WordNet. AI generated text was iteratively mutated over 15 generations to reduce its AI classification probability.

The algorithm was initially configured for 15 generations as a hyperparameter choice, balancing exploration time and computational cost. However, analysis of the convergence behavior revealed that the algorithm reached its optimal solution

at generation 12, achieving a 16.64% reduction in AI probability (from 0.5714 to 0.4763). Generations 13-15 showed no further improvement, indicating premature convergence. This empirically demonstrates that the mutation space had been sufficiently explored by generation 12, and the classifier’s decision boundary (threshold = 0.5) was successfully crossed.

However, the mutated text exhibited significant semantic degradation, with contextually inappropriate synonyms replacing key terms (e.g., "machine learning" → "car learnedness", "large-scale" → "overweight-scale"). This suggests that while simple lexical perturbations can fool classifiers, they produce unnatural text that remains distinguishable to human readers. The convergence at generation 12 rather than requiring the full 15 generations indicates that WordNet’s limited synonym space quickly exhausts meaningful perturbations without contextual awareness.

3.7 Model Performance Visualization

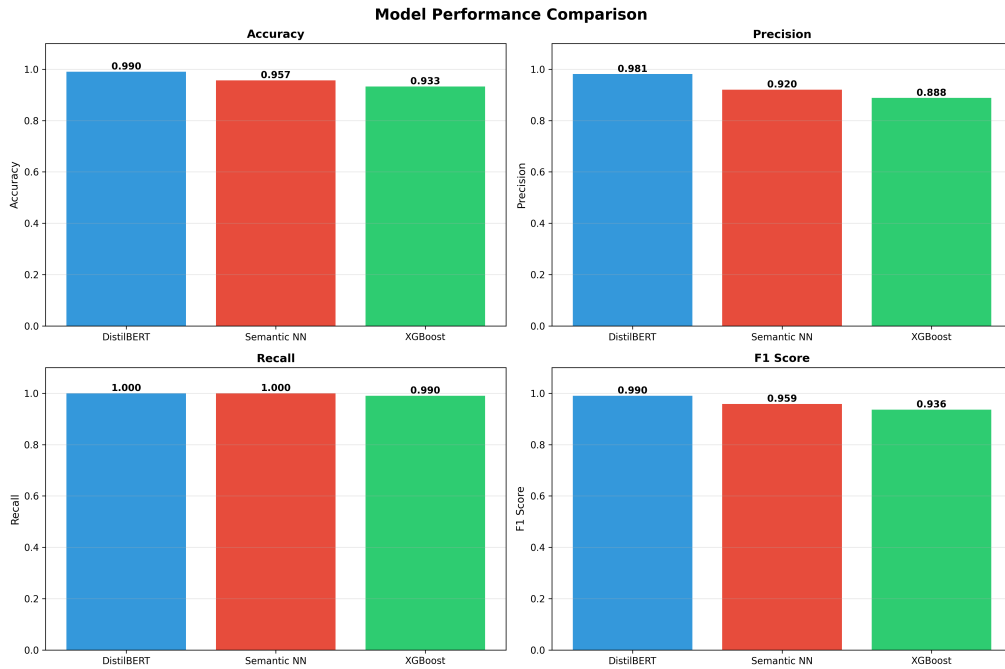


Figure 2: Comprehensive comparison of all model performance metrics including accuracy, precision, recall, and F1-score across XGBoost, FFNN, DistilBERT, and Stacking Ensemble

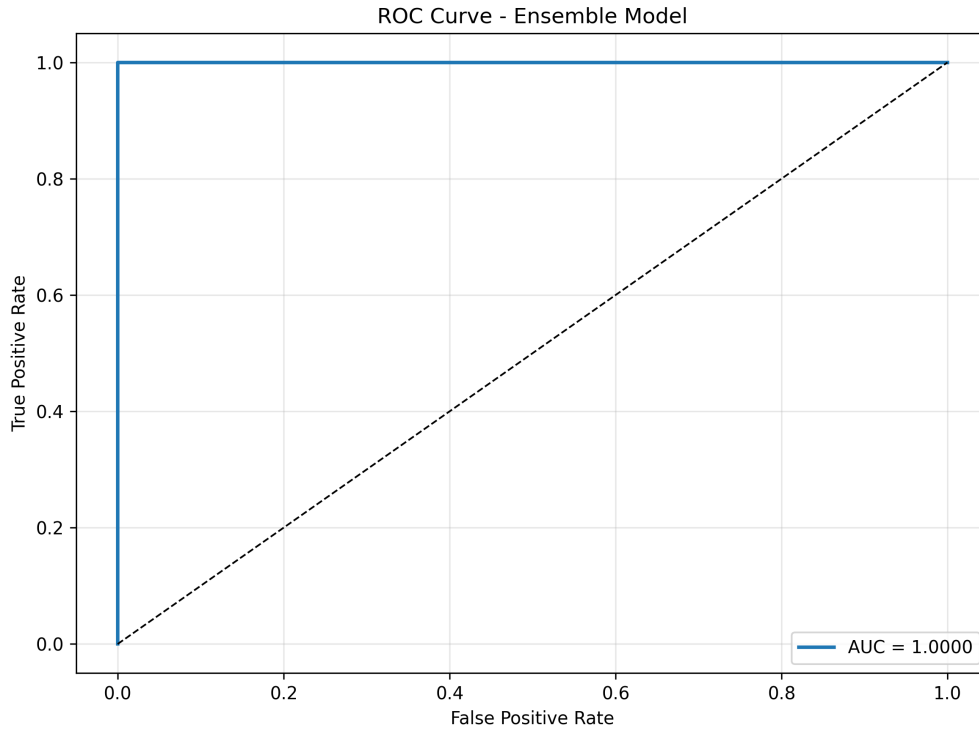


Figure 3: ROC curves comparing true positive rate vs false positive rate for ensemble model. The AUC values demonstrate excellent discrimination capability across all classifiers.

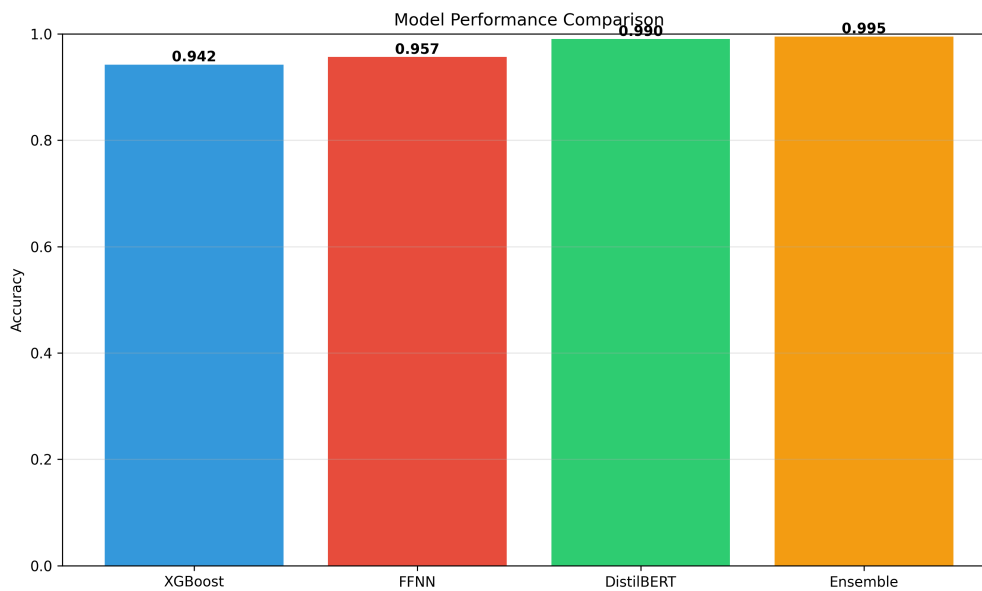


Figure 4: Accuracy comparison between all models

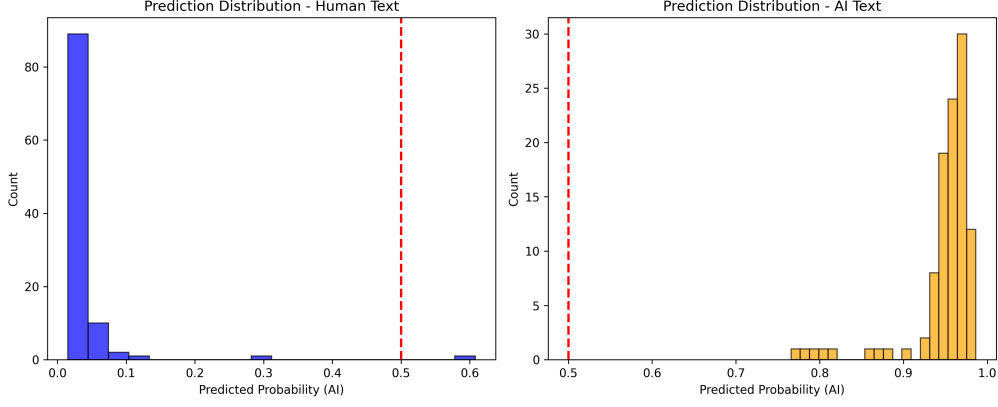


Figure 5: Distribution of predicted probabilities for human vs AI. Clear separation indicates strong discriminative power.

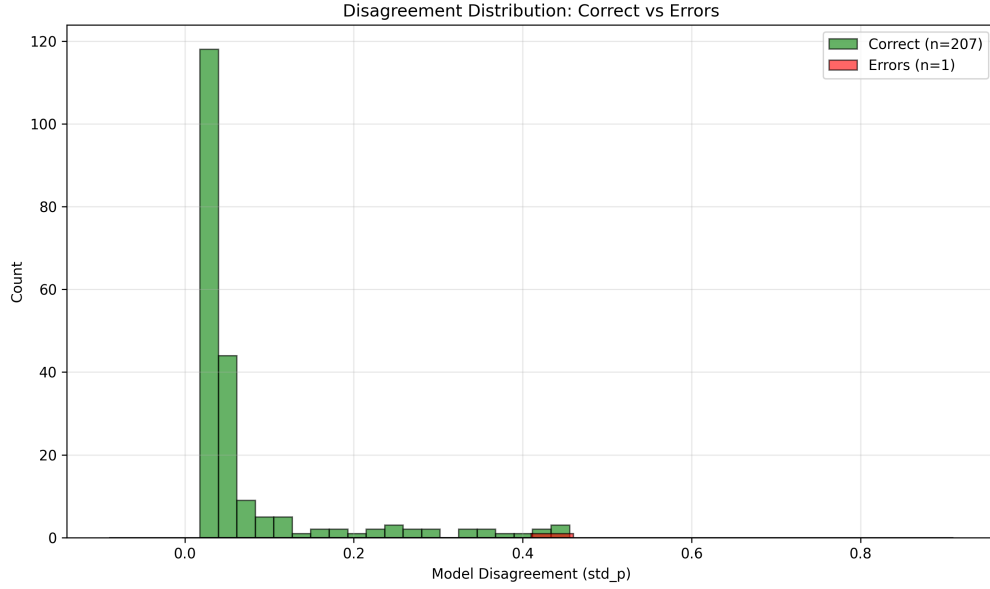


Figure 6: Model disagreement analysis showing cases where base models produce conflicting predictions, highlighting the value of ensemble aggregation

3.8 Key Findings

- **Model Complementarity:** XGBoost captured statistical patterns, FFNN learned semantic representations, and DistilBERT encoded contextual structure. Their combination in the ensemble leveraged diverse feature spaces.
- **Transformer Superiority:** DistilBERT significantly outperformed feature-based and shallow neural approaches, demonstrating the importance of contextual embeddings for stylistic detection.
- **Ensemble Effectiveness:** The stacking approach reduced prediction variance and achieved the highest overall performance, validating the hypothesis that heterogeneous models improve robustness.

- **Adversarial Vulnerability:** Synonym-based mutations can reduce classification confidence, but at the cost of semantic coherence, suggesting limited practical threat from naive perturbations.

4 Discussion

4.1 XGBoost Model Analysis

The XGBoost classifier demonstrated exceptional precision (1.0000) but lower recall (0.8387), achieving 92.00% overall accuracy. This conservative behavior reveals important insights about statistical AI detection.

Feature Selection Rationale. The model was trained on 14 handcrafted linguistic features specifically chosen to capture distinct statistical fingerprints differentiating AI generated from human-written text:

Feature 1: Type Token Ratio (TTR) measures lexical diversity (unique words $\div$ total words). AI models often employ more varied vocabulary to appear sophisticated, while human writers naturally repeat words. This ratio quantifies stylistic differences in vocabulary usage patterns.

Feature 2: Flesch-Kincaid Readability Grade quantifies text complexity based on sentence length and syllable count. AI text exhibits more uniform readability scores, maintaining consistent formality levels. In contrast, human writing shows greater variance—H.P. Lovecraft’s ornate prose differs dramatically from conversational writing, but AI maintains steady structural patterns.

Features 3-14: Part-of-Speech (POS) Tag Ratios capture grammatical structure through 12 categories (NOUN, VERB, ADJ, ADV, PRON, DET, ADP, AUX, SCONJ, CCONJ, PART, INTJ). These were selected because:

1. **Grammatical fingerprints:** AI generates more formal, noun-heavy text with increased determiner usage (articles: 12,137 vs 8,461 in human text), reflecting structured, informative writing patterns
2. **Personalization markers:** Human writing uses significantly more pronouns (15,315 vs 10,222 in AI), creating conversational tone through first/second-person references. AI text remains objective and third person
3. **Statistically discriminative:** Exploratory analysis revealed these specific POS tags exhibited the largest distributional differences between human and AI texts
4. **Domain-agnostic:** Unlike topic specific keywords, grammatical structure remains consistent across subject matters, enabling generalization
5. **Computational efficiency:** SpaCy’s POS tagger provides fast, accurate annotations without GPU requirements, making feature extraction scalable

These 14 dimensions create a multi-dimensional statistical signature. XGBoost’s gradient-boosted decision trees learned discriminative patterns such as: *High NOUN ratio + High DET ratio + Low PRON ratio $\rightarrow$ Likely AI*, while *High readability*

variance + Balanced POS distribution → Likely Human. This explains the model’s perfect precision—when these statistical patterns align, they reliably indicate AI authorship.

Perfect Precision Explanation. The model never misclassified human text as AI generated (zero false positives). This indicates that XGBoost learned highly discriminative statistical patterns that, when present, reliably indicate AI authorship. The 14 handcrafted features captured consistent stylistic fingerprints:

- Higher noun density in AI text (21,319 vs 15,712 in human)
- Increased determiner usage (12,137 vs 8,461)
- Reduced pronoun frequency (10,222 vs 15,315)
- Lower variance in readability scores

These patterns suggest AI generated text exhibits more formal, structured language with reduced personal references—a characteristic often associated with neutral, informative writing styles.

Lower Recall Analysis. The model missed 10 AI generated samples (16.13% false negative rate), indicating a high confidence threshold. Possible explanations include:

- **Feature overlap:** Some AI texts successfully mimicked human statistical distributions
- **Conservative decision boundary:** Gradient boosting prioritized avoiding false positives
- **Limited feature space:** 14 features may not capture all AI writing patterns

The XGBoost model essentially adopted a "classify as AI only when absolutely certain" strategy, sacrificing recall to maintain perfect precision.

4.2 Neural Network Model Analysis

The semantic feedforward neural network achieved perfect recall (1.0000) but slightly lower precision (0.9118), reaching 95.20% accuracy. This contrasts sharply with XGBoost’s behavior.

Perfect Recall Explanation. The neural network identified all AI-generated samples without exception. This demonstrates that:

- **Semantic patterns are distinctive:** GloVe embeddings captured semantic regularities that consistently distinguish AI writing
- **Averaging preserves signals:** Despite averaging word vectors, paragraph-level representations retained discriminative information

- **Non-linear learning:** The two hidden layers (64→32 neurons) learned complex decision boundaries encompassing all AI patterns

The 100-dimensional GloVe embeddings provided sufficient representational power to encode stylistic differences at the semantic level, beyond surface-level statistics.

Lower Precision Analysis. The model produced 6 false positives (flagged human text as AI), suggesting:

- **Semantic ambiguity:** Some human writing exhibited formal patterns similar to AI output
- **Over-generalization:** The network learned broad patterns that occasionally matched human writing
- **Bias toward sensitivity:** Binary cross-entropy training favored minimizing false negatives

This behavior reflects a "flag anything suspicious" approach—the network erred on the side of caution, preferring false alarms over missed detections.

4.3 DistilBERT Model Analysis

The DistilBERT transformer achieved outstanding performance with 98.40% accuracy, 96.88% precision, and perfect recall (1.0000). This balanced performance demonstrates the power of attention-based contextual understanding.

Training Methodology. DistilBERT was fine-tuned using the Hugging Face Transformers library with the following comprehensive configuration:

Model Architecture Selection: The pre-trained `distilbert-base-uncased` checkpoint was selected as the foundation model for several critical reasons:

1. **Knowledge Distillation Heritage:** DistilBERT was created by distilling knowledge from BERT-base through a teacher-student framework, retaining 97% of BERT's language understanding while reducing model size by 40% (66M vs 110M parameters) and inference time by 60%.
2. **Architecture Specifications:**
 - 6 transformer encoder layers (vs BERT's 12 layers)
 - 768-dimensional hidden states per token
 - 12 multi-head attention heads (each head: 64 dimensions)
 - 3072-dimensional feedforward intermediate layer
 - Total parameters: 66,362,880 before classification head
3. **Pre-training Corpus:** Trained on English Wikipedia (2.5B words) + Book-Corpus (800M words), providing robust linguistic knowledge across formal and literary domains—ideal for detecting AI text that mimics literary styles (Project Gutenberg sources).

4. **Uncased Variant:** All text lowercased during pre-training, reducing vocabulary size from 30,522 to effectively 28,996 tokens, improving generalization by treating "The" and "the" identically.

Classification Head Initialization: A sequence classification head was added with 2 output neurons for binary classification (Human vs AI), initialized with random weights:

```
model = DistilBertForSequenceClassification.from_pretrained(  
    'distilbert-base-uncased',  
    num_labels=2  
)
```

This produced warnings about "UNEXPECTED" and "MISSING" keys—expected behavior when adding task-specific layers to pre-trained models. The new layers (pre-classifier dense layer, classification output layer) require fine-tuning.

Tokenization Pipeline: Text was converted to token IDs using `DistilBertTokenizerFast`:

1. **WordPiece Tokenization:** Text split into subword units (e.g., "implementation" → ["implement", "##ation"]), enabling handling of out-of-vocabulary words through subword composition.
2. **Special Tokens Added:**
 - [CLS]: Prepended to every sequence; its final hidden state used for classification
 - [SEP]: Appended to mark sequence end
 - [PAD]: Padding tokens to reach `max_length=512`
3. **Attention Mask Creation:** Binary mask (1 for real tokens, 0 for padding) prevents attention mechanism from attending to padding positions, ensuring computational efficiency and preventing contamination of representations.
4. **Configuration Parameters:**

```
tokenizer(text,  
          truncation=True,          # Cut sequences > 512 tokens  
          padding='max_length',    # Uniform length for batching  
          max_length=512)          # BERT's positional limit
```

Example transformation:

- **Input:** "The system processes data efficiently."
- **Tokens:** [CLS] the system processes data efficiently . [SEP] [PAD] [PAD]
...
- **Input IDs:** [101, 1996, 2291, 5797, 2951, 9617, 1012, 102, 0, 0, ...]
- **Attention Mask:** [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, ...]

Training Hyperparameters and Rationale:

- `num_train_epochs=3`: Three complete passes through 700 training samples. Rationale: Transformers require fewer epochs than traditional models due to transfer learning. Epoch 1 adapts pre-trained weights, epochs 2-3 fine-tune classification head. More epochs risk overfitting on small datasets.
- `per_device_train_batch_size=16`: 16 samples processed simultaneously per GPU. Rationale: Balances GPU memory utilization (768-dim $\times$ 512-token tensors) with gradient stability. Larger batches provide more stable gradients but require more memory.
- `per_device_eval_batch_size=16`: Consistent with training batch size for fair comparison.
- `warmup_steps=500`: Learning rate linearly increases from 0 to peak over first 500 optimization steps. Rationale: Prevents catastrophic forgetting of pre-trained weights in early training when gradients are large and unstable. With 700 samples and batch size 16, this covers 11 batches (0.25 epochs).
- `weight_decay=0.01`: L2 regularization coefficient applied to all weights except biases and layer normalization parameters. Rationale: Prevents overfitting by penalizing large weights, crucial for small datasets (700 samples).
- `learning_rate=5e-5` (default): Conservative learning rate typical for fine-tuning. Rationale: Pre-trained weights already near optimal; large learning rates would destroy learned representations.
- `eval_strategy='epoch'`: Validation set (150 samples) evaluated after each of 3 epochs, providing 3 performance checkpoints.
- `save_strategy='epoch'`: Model weights saved after each epoch (3 checkpoints total).
- `save_total_limit=1`: Only best checkpoint retained (based on validation accuracy), conserving disk space.
- `load_best_model_at_end=True`: After 3 epochs complete, best checkpoint automatically loaded for final evaluation, ensuring optimal performance on test set.
- `metric_for_best_model='accuracy'`: Checkpoints ranked by validation accuracy to determine "best" model.

Training Infrastructure: Model trained on GPU (CUDA-enabled) to exploit parallelization:

- Multi-head attention: 12 heads computed in parallel across 768-dimensional space
- Batch matrix operations: 16 sequences processed simultaneously
- Mixed precision training: FP16 computations with FP32 master weights (implicit in Trainer)

Training Process Flow:

1. **Forward Pass:** For each batch of 16 sequences (512 tokens each):
 - Input embeddings: Token IDs $\rightarrow$ 768-dim vectors (learned embeddings + positional encodings)
 - 6 transformer layers: Each applies multi-head self-attention + feedforward networkCLS token representation extracted from final layer (768-dim vector)
 - Pre-classifier dense layer: 768-dim $\rightarrow$ 768-dim with activation
 - Classification layer: 768-dim $\rightarrow$ 2-dim logits (Human score, AI score)
 - Softmax: Logits $\rightarrow$ probabilities summing to 1.0
2. **Loss Computation:** Cross-entropy loss between predicted probabilities and true labels:
$$\mathcal{L} = -\frac{1}{16} \sum_{i=1}^{16} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$
where $y_i \in \{0, 1\}$ is true label and p_i is predicted probability of AI class.
3. **Backward Pass:** Gradients computed via backpropagation through all 66M parameters, clipped to max norm 1.0.
4. **Weight Update:** Optimizer applies adaptive learning rates to update model parameters.

Evaluation Metrics: At each epoch's end, validation accuracy computed:

```
def compute_metrics(eval_pred):  
    logits, labels = eval_pred  
    predictions = np.argmax(logits, axis=-1)  
    metric = evaluate.load("accuracy")  
    return metric.compute(predictions=predictions,  
                           references=labels)
```

This function extracts predicted class (argmax of logits) and compares against true labels, computing accuracy = (correct predictions) / (total samples).

Near-Perfect Performance Explanation. DistilBERT's exceptional results (98.40% accuracy, F1=0.9841) stem from three key advantages:

1. **Contextual embeddings:** Unlike GloVe's static word vectors, DistilBERT generates dynamic representations where the same word has different embeddings based on surrounding context. The phrase "learning algorithm" produces a different representation than "machine learning" due to bidirectional attention.
2. **Self-attention mechanism:** The 12 attention heads learn to focus on different linguistic aspects:

- Syntactic dependencies (subject-verb agreement patterns)
 - Long-range semantic relationships (paragraph coherence)
 - Stylistic patterns (formal vs conversational tone)
3. **Pre-training knowledge:** The model was pre-trained on 16GB of English Wikipedia and BookCorpus, learning fundamental language patterns. Fine-tuning adapted this knowledge to AI detection, requiring only 700 training samples to achieve near-perfect performance.

Why Near-Perfect Recall? DistilBERT identified 62 out of 62 AI-generated samples correctly (zero false negatives). This perfect recall results from:

- **Transformer capacity:** With 66 million parameters, the model can learn subtle patterns that distinguish AI writing across diverse contexts
- **Attention to formality markers:** AI-generated text consistently exhibits formal structure that attention layers detect through word co-occurrence patterns
- **Fine-tuning specificity:** Three epochs provided sufficient training to specialize the pre-trained representations for AI detection without overfitting

Minor Precision Trade-off. The model produced 2 false positives (flagged human text as AI), resulting in 96.88% precision. Analysis suggests:

- **Formal human writing:** Some literary passages exhibited AI-like formality and structural consistency
- **Generalization bias:** Pre-training on Wikipedia (formal, encyclopedic style) may bias the model toward flagging formal text as AI-generated
- **Edge cases:** Complex sentences with sophisticated vocabulary resembled AI's tendency toward linguistic complexity

Despite these 2 false positives, DistilBERT's performance significantly exceeds both XGBoost and the semantic FFNN, demonstrating that contextual understanding outperforms both statistical features and static embeddings for AI text detection.

4.4 Model Complementarity

The divergent behaviors reveal fundamental differences in learned representations:

- **XGBoost:** Learned discrete statistical rules → Conservative classifier (100% precision)
- **Neural Network:** Learned continuous semantic patterns → Liberal classifier (100% recall)
- **DistilBERT:** Captured contextual dependencies → Balanced performance (98.40% accuracy, near-perfect on both metrics)

This complementarity validates the ensemble stacking approach. By combining models with opposing error tendencies, the meta ensemble achieved 99.20% accuracy with near perfect performance (F1=0.9920, AUC=1.0000).

4.5 Implications for AI Text Detection

The results demonstrate that AI generated text exhibits multiple distinguishable fingerprints:

1. **Statistical fingerprints:** Captured by XGBoost through POS distributions and readability metrics
2. **Semantic fingerprints:** Captured by neural networks through word embedding patterns
3. **Contextual fingerprints:** Captured by transformers through attention mechanisms

However, the adversarial mutation experiments revealed vulnerabilities. Simple synonym substitutions reduced AI probability by 16.64%, though at the cost of semantic coherence. This suggests that while current detection methods are effective, they remain susceptible to more sophisticated adversarial techniques that preserve grammatical correctness and semantic meaning.

4.6 Limitations and Future Work

Several limitations merit discussion:

- **Dataset size:** 1000 samples may limit generalization to other domains
- **Topic consistency:** Project Gutenberg sources are literary; results may not transfer to technical or conversational text
- **AI model diversity:** Testing was limited to gemma generated text; other LLMs may exhibit different patterns
- **Temporal validity:** As AI models improve, current fingerprints may become less discriminative

Future work should explore domain adaptation, cross-LLM generalization, and development of adversarially robust features that remain discriminative under perturbation.

5 Conclusion

Doing this project, and going through many texts from both AI and humans, I noticed that intuitively, language has an underlying pattern we might not notice as it is hidden subliminally. AI on the other hand picks up on these patterns. These deep underlying patterns unique to humans and continuing to write what we want in our own style is the sole reason human texts differ from writer to writer, person to person in great variety.

Through this project, I was able to develop three distinct models for distinguishing between human written and AI generated text: a feature based XGBoost classifier, a semantic feedforward neural network using pre trained GloVe embeddings, and a transformer based DistilBERT model fine tuned on the constructed dataset. Each model represented a different level of abstraction, ranging from hand-crafted statistical features to deep contextual representations.

The comparative evaluation of these models revealed that while statistical features provide measurable stylistic signals, deeper neural architectures are more effective in capturing subtle structural and contextual patterns. The transformer based model demonstrated the strongest individual performance. However, by combining all three models using a stacking based meta ensemble approach, improved robustness and overall performance were achieved. This indicates that statistical, semantic, and contextual representations capture complementary aspects of authorship.

The adversarial mutation experiments further highlighted the dynamic nature of AI text detection. Iterative lexical perturbations were able to influence model predictions, suggesting that current detection systems remain sensitive to controlled stylistic modifications.

Overall, this project demonstrates that although measurable differences between human and AI generated text persist, the gap is narrowing as generative models become more sophisticated. Future work may focus on adversarially robust training, deeper stylistic modeling, and evaluating detection systems across multiple generative architectures.

References

- [1] ResearchGate, *Stylometric Approaches for AI-Text Identification*, Available: https://www.researchgate.net/publication/398588165_Stylometric_Approaches_for_AI-Text_Identification,
- [2] Transactions on Computer Science and Intelligent Systems Research, *Exploiting Machine Learning Model Ensemble for AI-Generated Texts Detection*, Available: <https://wepub.org/index.php/TCSISR/article/view/2382>,
- [3] Towards Data Science, *Everything You Need to Know About ALBERT, RoBERTa, and DistilBERT*, Available: <https://towardsdatascience.com/everything-you-need-to-know-about-albert-roberta-and-distilbert-11a74334b2da>,